

Set Cover

Решение: отжиг

Состояние

- Маппинг element -> set его покрывающий + обратный маппинг
- cost: сумма весов элементов

Изменения

- Удалить случайное множество и заменить элементы которые им покрывались на другие случайные множества (отсэмплировав их по функции полезности, в начале была 1).

Первая итерация

Шедулинг отжига: $T_{\text{next}} = T_{\text{prev}} \cdot 0.999999$, $P(\text{переход}) = \exp\left(-\frac{\text{score}_{\text{current}}}{\text{score}_{\text{candidate}} \cdot T}\right)$.

Instance	Cost score	CPU time, s	Points
sc_157_0	95 800	6.01	3
sc_330_0	30	212.58	0
sc_1000_11	156	48.09	3
sc_5000_1	40	81.39	3
sc_10000_5	94	93.69	3
sc_10000_2	218	41.32	3

Итого: 15 / 30.

Вторая итерация

Более классическая формула для вероятности принятия неоптимального состояния:

$$\exp\left(-\frac{\Delta \text{score}}{T}\right)$$

Instance	Cost score	CPU time, s	Points
sc_157_0	101 600	5.16	3
sc_330_0	23	72.42	5
sc_1000_11	150	15.24	3
sc_5000_1	31	99.99	5
sc_10000_5	83	109.19	3
sc_10000_2	196	42.83	3

Итого: 22 / 30.

Третья итерация

Нормализованы косты множеств для отжига, чтобы в среднем кост был 1.

Instance	Cost score	CPU time, s	Points
sc_157_0	94 600	5.27	3
sc_330_0	23	66.84	5
sc_1000_11	149	14.41	3
sc_5000_1	31	96.01	5
sc_10000_5	92	110.69	3
sc_10000_2	195	45.62	3

Итого: 22 / 30.

Четвёртая итерация

Поменял шедулер температуры с экспоненциального на reciprocal, который вначале остывает быстро, потом медленнее: $T_{it} = \frac{T_0}{1 + rate \cdot it}$

Instance	Cost score	CPU time, s	Points
sc_157_0	95 400	6.18	3
sc_330_0	26	70.07	3
sc_1000_11	149	17.00	3
sc_5000_1	42	105.01	3
sc_10000_5	76	117.34	3
sc_10000_2	174	48.32	3

Итого: 18 / 30.

Пятая итерация

Поменял шедулер температуры с reciprocal на линейный, который остывает равномерно: $T_{it} = T_0 - decrement \cdot it$

Instance	Cost score	CPU time, s	Points
sc_157_0	94 600	6.15	3
sc_330_0	23	69.22	5
sc_1000_11	162	16.41	3
sc_5000_1	45	93.02	3
sc_10000_5	144	106.38	0
sc_10000_2	276	48.68	3

Итого: 17 / 30. Откатил шедулер на экспоненциальный.

Шестая итерация

При замене множества на новое семплирую с весом “полезности” множества – сумме $\frac{1}{cnt}$ по всем элементам множества деленной на цену этого множества, где cnt – число множеств, покрывающих элемент.

Семплирование делаю с помощью дерева Фенвика.

Instance	Cost score	CPU time, s	Points
sc_157_0	94 600	4.52	3
sc_330_0	23	63.98	5
sc_1000_11	146	12.33	5
sc_5000_1	31	82.28	5
sc_10000_5	67	95.34	3
sc_10000_2	167	42.94	5

Итого: 26 / 30.

Седьмая итерация

Увеличил число итераций в два раза, поменяв множитель температуры на 0.9999995.

Instance	Cost score	CPU time, s	Points
sc_157_0	94 400	8.85	5
sc_330_0	23	129.35	5
sc_1000_11	146	24.82	5
sc_5000_1	31	164.92	5
sc_10000_5	65	186.42	3
sc_10000_2	167	83.16	5

Итого: 28 / 30.

Восьмая итерация

Добавил случайный шаг нормализации: элементы переназначаются на первое активное множество из случайного порядка.

Instance	Cost score	CPU time, s	Points
sc_157_0	94 600	9.49	3
sc_330_0	23	146.15	5
sc_1000_11	146	28.62	5
sc_5000_1	30	181.18	5
sc_10000_5	65	206.25	3
sc_10000_2	167	91.74	5

Итого: 26 / 30. Лучше не стало, откатил изменение.

Десятая итерация

Добавил аналогичную utility-функцию, по которой семплирую множества для удаления: чем меньше необычных элементов добавляет множество, тем чаще мы его удаляем.

Проверил стабильность: запустил на трёх случайных седах каждый тест, оцениваю по медиане (сиды: 2926824233, 3832009357, 4231247517).

Instance	Score 1	Score 2	Score 3	Mean CPU time, s	Points
sc_157_0	94 600	94 600	94 600	7.54	3
sc_330_0	23	23	23	129.57	5
sc_1000_11	146	146	146	22.60	5
sc_5000_1	30	31	30	192.78	5
sc_10000_5	64	64	64	216.80	5
sc_10000_2	167	169	167	87.06	5

Итого: 28 / 30.